

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



NGHIÊN CỨU CẢI THIỆN
THUẬT TOÁN TỰ ĐỘNG XÁC ĐỊNH VỊ TRÍ
VÀ DỰNG BẢN ĐỒ 2D CHO
MÔ HÌNH MOBILE ROBOT TRONG NHÀ

BÁO CÁO HỌC PHẦN
LẬP TRÌNH ROBOT VỚI ROS
Ngành: Kỹ thuật Robot

HÀ NỘI – 2023

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

NGHIÊN CỨU CẢI THIỆN
THUẬT TOÁN TỰ ĐỘNG XÁC ĐỊNH VỊ TRÍ
VÀ DỰNG BẢN ĐỒ 2D CHO
MÔ HÌNH MOBILE ROBOT TRONG NHÀ

Thành viên nhóm: Nguyễn Huyền Trang - 20020727

Lê Phần Nam - 20020696

Cán bộ hướng dẫn: PGS.TS. Hoàng Văn Xiêm

KS. Nguyễn Cảnh Thanh

HÀ NỘI - 2023

TÓM TẮT

Tóm tắt:

Đề tài được triển khai với mục tiêu có thể giải quyết bài toán tự động định vị và dựng bản đồ phẳng môi trường hai chiều cho robot di động trong nhà - xây dựng bài toán SLAM (Simultaneous Localization and Mapping) trên nền tảng môi trường ROS Noetic. Quá trình mô phỏng hoạt động của robot được thực hiện trên hai phần mềm Gazebo và RViz để trực quan hóa dữ liệu kết quả thu được. Bài toán sử dụng mô hình robot TurtleBot 3 model Burger hoặc Waffle với các loại cảm biến được xây dựng sẵn trên mô hình để thu nhận tín hiệu từ môi trường và sử dụng cảm biến Lidar là sensor chính để thu dữ liệu tạo dựng bản đồ.

Bài toán thực hiện giải quyết vấn đề tự động định vị cho mô hình robot, đồng thời sử dụng package Gmapping để xây dựng bản đồ bằng phương pháp xử lý dữ liệu đo khoảng cách từ laser lidar và dữ liệu từ odometry và tập hợp ghép nối nhiều lớp bản đồ mà robot nhận được trong quá trình dịch chuyển trong môi trường. Trong quá trình thực hiện bài toán, đề tài có triển khai một số phương hướng cải thiện mô hình SLAM để tối ưu hơn về quãng đường dịch chuyển khám phá trong môi trường chưa xác định của robot, đồng thời tinh chỉnh một số thông số đầu vào để tối ưu hóa phương án thực hiện.

Từ khóa: Robot Operating Systems (ROS), Simultaneous Localization and Mapping (SLAM), RViz, Gazebo, Gmapping, TurtleBot3, Noetic, rqt, Collision-Free Autonomous Navigation.

MỤC LỤC

DANH MỤC HÌNH ẢNH	4
MỞ ĐẦU	1
CHƯƠNG 1: TỔNG QUAN	2
1.1. Robot và sự phát triển trong công nghiệp	2
1.2. Mobile Robot – Sự ra đời và tầm quan trọng trong kỹ thuật	2
1.3. Giới thiệu về SLAM	4
1.4. Hệ điều hành cho robot	6
1.4.1. Đặc điểm của ROS	6
1.4.2. Tiếp cận ROS như thế nào	7
1.4.3. Tại sao lựa chọn ROS	9
1.5. Đặt vấn đề nghiên cứu	10
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ MÔ HÌNH ROBOT	11
2.1. Cơ sở lý thuyết	11
2.1.1. LiDAR SLAM	11
2.1.2. Thuật toán Gmapping	13
2.1.3. Đóng vòng lặp	14
2.2. Mô hình robot hai bánh vi sai	15
CHƯƠNG 3: XÂY DỰNG CHƯƠNG TRÌNH	17
3.1. Mục tiêu đạt được	17
3.2. Đề xuất phương án thực hiện	17
3.2.1. Mô phỏng động lực học cho robot.	17
3.2.2. Dựng bản đồ môi trường.	17
3.2.3. Định vị và điều hướng robot khám phá môi trường.	18
3.3. Triển khai bài toán	19
3.3.1. Dựng mô hình mô phỏng robot.	19
3.3.2. Thu bản đồ từ Gmapping:	20
3.3.3. Truyền quỹ đạo vào mô hình robot	21

3.4. Quá trình chạy chương trình.	21
3.4.1. Khởi tạo môi trường làm việc	21
3.4.2. Chạy chương trình mô phỏng cho robot.	23
3.4.3. Một số thao tác lấy dữ liệu trong quá trình thực hiện.	25
3.5. Tiêu chí cải thiện mô hình.	26
3.5.1. Inflation_radius	26
3.5.2. Thay đổi tham số đầu vào.	27
3.5.3. Tối ưu hóa quãng đường thực hiện.	27
CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ	29
4.1. Testcase 1: Môi trường đơn giản.	29
4.1.1. Mức tiếp cận cơ bản.	29
4.1.2. Cải thiện chương trình.	29
4.2. Testcase 2: Môi trường bản đồ mê cung.	30
4.2.1. Mức tiếp cận cơ bản.	30
4.2.2. Cải thiện chương trình.	31
4.3. Testcase 3: Môi trường bản đồ trong phòng.	32
4.3.1. Mức tiếp cận cơ bản.	32
4.3.2. Cải thiện chương trình.	34
CHƯƠNG 5: TỔNG KẾT	37
TÀI LIỆU THAM KHẢO	38

DANH MỤC HÌNH ẢNH

Hình 1.1. Robot hàn trong công nghiệp.	2
Hình 1.2. Mobile robot phục vụ trong kho hàng.	3
Hình 1.3. Sơ đồ hệ thống dẫn đường cho robot di động.	4
Hình 1.4. Front-end và back-end trong hệ thống SLAM.	5
Hình 1.5. ROS (Robot Operating System) là gì.	7
Hình 1.6. Tổ chức dữ liệu đơn giản trong ROS.	8
Hình 1.7. Giao diện phần mềm Gazebo và phần mềm RViz.	9
Hình 2.1. Cảm biến LiDAR.	12
Hình 2.2. Cách triển khai thuật toán Gmapping SLAM trong robot di động.	13
Hình 2.3. Minh họa so sánh giữa bản đồ khi có/không sử dụng đóng vòng lặp.	15
Hình 2.4. Mô hình robot di động hai bánh vi sai.	15
Hình 3.1. Khởi chạy môi trường thứ nhất TB3_Hexa.world.	19
Hình 3.2. Khởi chạy môi trường thứ hai maze.world.	19
Hình 3.3. Khởi chạy môi trường thứ ba myhouse.world.	20
Hình 3.4. Các file .launch đã tạo tùy chỉnh cho từng môi trường.	20
Hình 3.5. Phạm vi tiếp cận của robot sau và trước khi điều chỉnh inflation_radius.	27
Hình 3.6. Quá trình tạo quãng đường tối ưu tự động của robot.	28
Hình 4.1. Bản đồ môi trường TB3.world.	29
Hình 4.2 Bản đồ môi trường TB3.world với chương trình cải thiện.	30
Hình 4.3. Bản đồ môi trường maze.world.	31
Hình 4.4. Bản đồ môi trường maze.world.	32
Hình 4.5. Bản đồ môi trường myhouse.world.	33
Hình 4.6. Quỹ đạo robot tạo ra để ra khỏi phòng.	34
Hình 4.7. Bản đồ môi trường myhouse.world.	35
Hình 4.8. Bản đồ môi trường myhouse.world.	35
Hình 4.9. Bản đồ môi trường lần chạy thứ tư.	36
Hình 4.10. Kết quả bản đồ cuối cùng sau khi chạy chương trình tối ưu.	36

MỞ ĐẦU

Trong thời đại cách mạng công nghiệp 4.0 hiện nay, tự động hóa và robot đã trở thành xu hướng phổ biến. Điều này đặc biệt quan trọng trong các ngành sản xuất đòi hỏi độ chính xác cao như: sản xuất linh kiện điện tử, lắp ráp ô tô, sản xuất thuốc và thực phẩm, và nhiều ứng dụng khác. Một trong những khía cạnh quan trọng nhất của robot là khả năng tương tác với con người. Tương tác người và robot mang lại nhiều lợi ích, từ việc giúp con người thực hiện các công việc phức tạp và nguy hiểm, đến việc tạo ra một môi trường làm việc hiệu quả và thuận tiện. Báo cáo này gồm 5 chương:

Chương 1. Tổng quan.

Chương 1: Mô tả giới thiệu chung về xu hướng phát triển trong mảng robot di động và nhu cầu cấp thiết cần xây dựng bài toán tự động định vị và lập bản đồ cho robot, giới thiệu về các khái niệm cần thiết cho bài toán SLAM, phân loại và ứng dụng và môi trường nhóm dự định thực hiện bài toán này trên nền tảng ROS.

Chương 2. Cơ sở lý thuyết và mô hình robot.

Chương 2: Trình bày nền tảng kiến thức của bài toán SLAM và các thuật toán được sử dụng trong quá trình xây dựng mô hình tự động định vị và lập bản đồ của nhóm.

Chương 3. Xây dựng chương trình.

Chương 3: Trình bày quá trình xây dựng chương trình của nhóm đã thực hiện. Gồm có mục tiêu đầu ra của bài toán, các hướng triển khai xây dựng chương trình, đánh giá quá trình thực hiện và đề xuất phương án cải thiện cho mô hình.

Chương 4. Kết quả và đánh giá

Chương 4: Tổng hợp toàn bộ kết quả đã làm được sau quá trình triển khai thực hiện xây dựng và cải thiện mô hình giải quyết bài toán tự động định vị và xây dựng bản đồ không gian hai chiều cho robot di động trong nhà sử dụng cảm biến LIDAR.

Chương 5. Tổng kết

Chương 5: Kết luận chung sau quá trình thực hiện bài toán và hướng phát triển thêm cho đề tài này trong tương lai.

CHƯƠNG 1: TỔNG QUAN

1.1. Robot và sự phát triển trong công nghiệp

Robot công nghiệp là các thiết bị tự động được thiết kế để thực hiện các tác vụ sản xuất trong môi trường công nghiệp. Robot hiện nay đã trải qua sự phát triển đáng kể, từ những máy móc cồng kềnh chỉ có khả năng thực hiện các tác vụ cơ bản đến những thiết bị thông minh, linh hoạt và có khả năng tương tác với môi trường xung quanh.

Các loại robot công nghiệp hiện nay rất đa dạng, từ robot hàn, robot gắn linh kiện, robot sơn, robot đóng gói, đến robot thao tác và vận chuyển hàng hóa. Chúng được lắp đặt và vận hành trong các dây chuyền sản xuất, trang thiết bị đóng gói và xử lý hàng hóa trong các nhà máy và xưởng sản xuất.



Hình 1.1. Robot hàn trong công nghiệp.

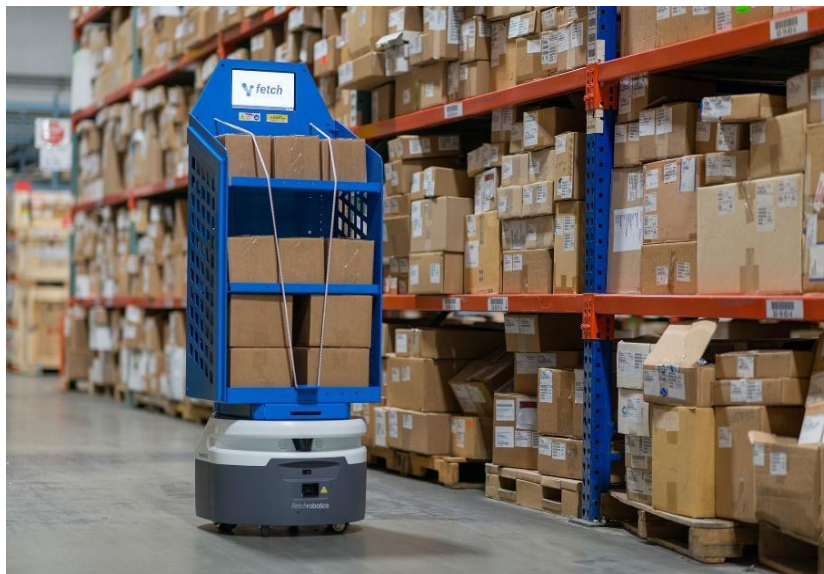
Các loại robot hiện nay bao gồm robot công nghiệp, robot y tế, robot dịch vụ và mobile robot.

1.2. Mobile Robot – Sự ra đời và tầm quan trọng trong kỹ thuật

Mobile robot là một loại robot di động có khả năng tự điều hướng và thực hiện các tác vụ trong môi trường không gian rộng. Chúng được thiết kế để di chuyển từ vị trí này sang vị trí khác, thường sử dụng bánh xe, chân robot hoặc các hệ thống truyền động khác để di chuyển trên mặt đất, trong nhà hoặc ngoài trời.

Có một lịch sử dài từ những năm 1960 đến hiện tại, mobile robot đã trải qua các giai đoạn phát triển đáng kể. Từ những robot chỉ có thể điều khiển bằng tay, hiện nay, mobile robot trở nên thông minh, linh hoạt và có khả năng học và thích ứng với môi trường. Các công nghệ như học máy, thị giác máy tính ... được tích hợp vào mobile robot, mở ra nhiều cơ hội ứng dụng trong các lĩnh vực công nghiệp, dịch vụ và hàng tiêu dùng. Mobile robot đóng vai trò quan trọng trong tương tác người-máy, với khả năng giao tiếp thông qua các giao diện người dùng, khả năng di chuyển tự động, xử lý thông tin và thực hiện các nhiệm vụ khác nhau.

Ngày nay, mobile robot có thể tự di chuyển trong môi trường mà không cần sự can thiệp của con người. Chúng có thể tự định vị, đo đạc và thu thập thông tin từ môi trường xung quanh để đưa ra quyết định và thực hiện các nhiệm vụ cần thiết. Với khả năng tự học và thích ứng, mobile robot có thể tương tác thông minh với môi trường và thích ứng với sự biến đổi của nó trong thời gian dài. Điều này mở ra nhiều cơ hội ứng dụng trong các lĩnh vực công nghiệp, dịch vụ và hộ gia đình.



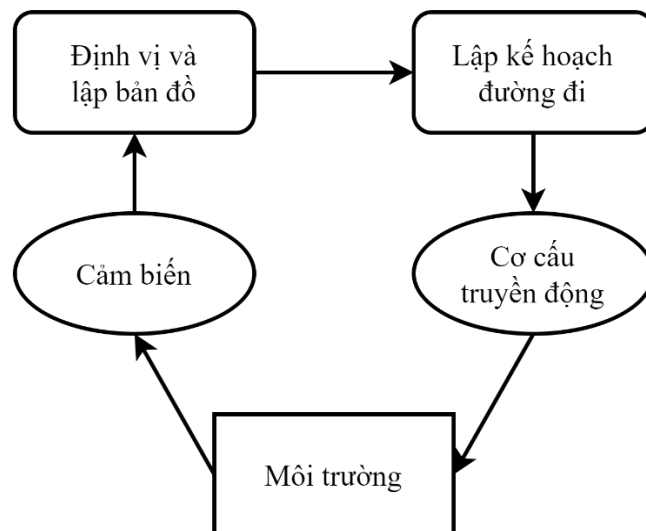
Hình 1.2. Mobile robot phục vụ trong kho hàng.

Mobile robot có thể hoạt động trong các môi trường đa dạng như nhà máy công nghiệp, sảnh tòa nhà, không gian công cộng và nhiều nơi khác, mang lại hiệu quả và tiện ích lớn. Chúng đóng góp không nhỏ vào sự phát triển và tiện ích của cuộc sống hàng ngày, giúp tăng cường hiệu suất làm việc, cung cấp sự linh hoạt và hiệu quả trong quản

lý và vận hành quy trình tự động.

Để robot có thể tự di chuyển trong môi trường, cần có một hệ thống dẫn đường và định hướng. Hệ thống này giúp robot xác định vị trí hiện tại và lập kế hoạch đường đi đến các vị trí mục tiêu. Quá trình dẫn đường bao gồm ba chức năng cơ bản: tự định vị, xây dựng bản đồ và lập kế hoạch đường đi.

Tự định vị cho phép robot biết vị trí của nó trong hệ quy chiếu của môi trường. Xây dựng bản đồ cho phép robot tạo ra một bản đồ chi tiết về môi trường xung quanh, bao gồm các vật cản và đặc điểm quan trọng. Lập kế hoạch đường đi là quá trình tìm ra đường đi an toàn và hiệu quả từ vị trí hiện tại đến vị trí mục tiêu dựa trên thông tin về bản đồ và các ràng buộc cụ thể. Hình 1.3 thể hiện sơ đồ hệ thống dẫn đường cho robot di động



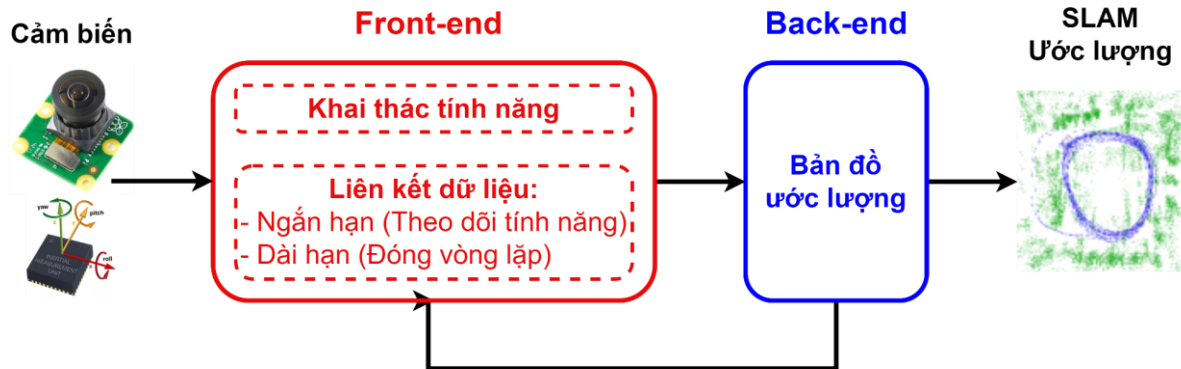
Hình 1.3. Sơ đồ hệ thống dẫn đường cho robot di động.

1.3. Giới thiệu về SLAM

SLAM là viết tắt của "Simultaneous Localization and Mapping", trong tiếng Việt có thể dịch là "Vị trí và Xây dựng bản đồ đồng thời". Đây là một công nghệ được sử dụng trong lĩnh vực robot tự hành cho phép một robot hoặc một hệ thống robot xây dựng bản đồ của môi trường và đồng thời xác định vị trí của chính nó trong môi trường đó.

Thuật toán SLAM tích hợp các thông tin từ các cảm biến như camera, cảm biến

lida, máy quét 3D và bộ cảm biến khác để xác định vị trí tương đối của robot và xây dựng một bản đồ môi trường. Đồng thời, nó cũng sử dụng thông tin này để cải thiện độ chính xác của vị trí và bản đồ.



Hình 1.4. Front-end và back-end trong hệ thống SLAM.

Để giải quyết bài toán SLAM, cần có hai công nghệ chính: front-end và back-end. Front-end là xử lý dữ liệu từ cảm biến đầu vào, như camera, laser, sonar, v.v. để trích xuất các thông tin về môi trường và vị trí của robot. Back-end là tối ưu hóa vị trí và bản đồ dựa trên các thông tin từ front-end.

Tùy theo loại cảm biến đầu vào và chiều không gian bản đồ đầu ra, SLAM có thể được phân loại theo nhiều cách khác nhau. Một số phương pháp SLAM phổ biến phân loại theo cảm biến gồm LiDAR SLAM, Visual SLAM.

LiDAR SLAM: Sử dụng cảm biến laser để đo khoảng cách từ robot đến các vật thể trong môi trường. Phương pháp này có độ chính xác cao, nhưng khó xử lý trong môi trường thiếu kết cấu và dễ bị sai lệch khi robot di chuyển.

Visual SLAM (hay vSLAM): Sử dụng camera để trích xuất các điểm đặc trưng từ các ảnh của môi trường và từ đó ước lượng vị trí của robot và bản đồ môi trường. Phương pháp này có ưu điểm là dữ liệu nhỏ gọn, dễ xử lý trong môi trường có kết cấu, nhưng không chính xác bằng LiDAR.

Visual-inertial SLAM: Là hệ thống visual SLAM nhưng được bổ sung thêm thông tin từ cảm biến gia tốc IMU để tăng độ chính xác của mô hình SLAM2.

Ngoài ra, SLAM còn được phân loại theo chiều không gian của bản đồ môi trường

là 2D hoặc 3D. 2D SLAM chỉ thể hiện vị trí (x, y) và hướng (yaw) của robot trong mặt phẳng (Oxy). 3D SLAM thể hiện thêm chiều cao, tạo bản đồ với sáu bậc tự do (6 DOF) gồm 3 bậc vị trí (x, y, z) và 3 hướng (roll, pitch, yaw).

SLAM là một bài toán khó vì nó đòi hỏi xử lý đồng thời các tác vụ vị trí và xây dựng bản đồ trong thời gian thực, bên cạnh đó cũng cần phải xử lý nhiễu từ cảm biến và môi trường. SLAM đã tạo ra sự tiến bộ đáng kể trong lĩnh vực robot tự hành và đã có nhiều ứng dụng thực tế như xe tự lái và robot dịch vụ. Phát triển SLAM đóng vai trò quan trọng trong việc cung cấp cho robot khả năng tự điều hướng và hiểu biết về môi trường xung quanh.

1.4. Hệ điều hành cho robot

Hệ điều hành cho robot là một hệ thống phần mềm cho phép robot thực hiện các nhiệm vụ khác nhau trong môi trường đa dạng và phức tạp. Hệ điều hành cho robot cung cấp các tính năng như:

Trừu tượng hóa phần cứng: Cho phép robot sử dụng các thiết bị và cảm biến khác nhau mà không cần quan tâm đến chi tiết kỹ thuật của chúng.

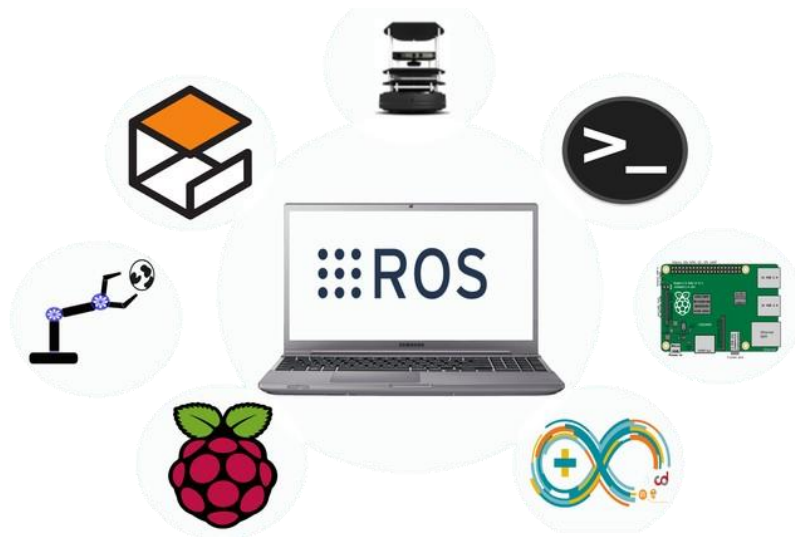
Quản lý tài nguyên: Cho phép robot phân bổ và sử dụng hiệu quả các tài nguyên như CPU, bộ nhớ, pin, v.v.

Giao tiếp: Cho phép robot trao đổi thông tin với các robot khác hoặc với con người qua các giao thức và giao diện tiêu chuẩn.

Lập trình: Cho phép robot thực hiện các thuật toán và ứng dụng được viết bằng các ngôn ngữ lập trình khác nhau.

Một trong những hệ điều hành cho robot phổ biến hiện nay là ROS (Robot Operating System). ROS là một nền tảng mã nguồn mở được phát triển bởi Willow Garage và Open Source Robotics Foundation. ROS cung cấp các thư viện và công cụ để giúp các nhà phát triển phần mềm tạo ra các ứng dụng cho robot.

1.4.1. Đặc điểm của ROS



Hình 1.5. ROS (Robot Operating System) là gì.

Không phải là một hệ điều hành thực sự: ROS không quản lý trực tiếp các tài nguyên của robot, mà chỉ là một lớp trung gian giữa hệ điều hành thực sự (như Linux) và các ứng dụng của robot.

Phân tán và song song: ROS không chạy trên một máy tính duy nhất, mà có thể chạy trên nhiều máy tính được kết nối qua mạng. Các ứng dụng của robot được chia thành nhiều tiến trình gọi là node, có thể chạy song song và giao tiếp với nhau qua tin nhắn gọi là topic.

Đa nền tảng và đa ngôn ngữ: ROS có thể chạy trên nhiều hệ điều hành khác nhau, như Linux, Windows, MacOS, v.v. ROS cũng hỗ trợ nhiều ngôn ngữ lập trình khác nhau, như C++, Python, Java, Lisp, v.v.

Mô-đun và tái sử dụng: ROS được tổ chức theo kiểu gói (package), mỗi gói chứa một hoặc nhiều node liên quan đến một chức năng cụ thể của robot. Các gói có thể được sử dụng lại cho nhiều loại robot khác nhau hoặc kết hợp với nhau để tạo ra các ứng dụng mới.

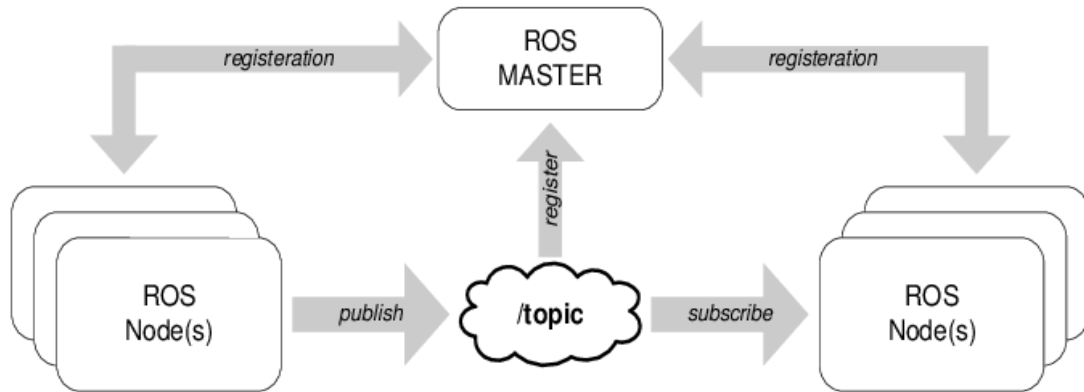
1.4.2. Tiếp cận ROS như thế nào

Cài đặt ROS: Để sử dụng ROS, chúng ta cần cài đặt phiên bản phù hợp với hệ điều hành và loại robot của mình. Hiện nay, có nhiều phiên bản ROS khác nhau, như Kinetic Kame, Melodic Morenia, Noetic Ninjemys, v.v. Mỗi phiên bản có những tính

năng và yêu cầu khác nhau.

Tạo workspace: Chúng ta tạo một workspace là một thư mục chứa các gói và mã nguồn của ứng dụng. Workspace giúp quản lý các gói và biên dịch chúng.

Tạo gói: Để tạo một ứng dụng với ROS, chúng ta cần tạo ít nhất một gói trong workspace. Một gói là một đơn vị tổ chức của ROS, chứa các node, topic, message, service, launch file và các tệp khác liên quan.



Hình 1.6. Tổ chức dữ liệu đơn giản trong ROS.

Viết node: Để viết một node trong ROS, chúng ta cần sử dụng một ngôn ngữ lập trình được hỗ trợ bởi ROS, như C++ hoặc Python. Một node là một tiến trình thực hiện một công việc cụ thể của robot. Mỗi node có một tên duy nhất và có thể giao tiếp với các node khác qua topic.

Viết topic: Để viết một topic trong ROS, chúng ta cần định nghĩa kiểu dữ liệu của tin nhắn được gửi qua topic. Một topic là một kênh giao tiếp giữa các node trong ROS. Mỗi topic có một tên duy nhất và có thể có nhiều node đăng ký (subscribe) hoặc xuất bản (publish) tin nhắn qua topic đó.

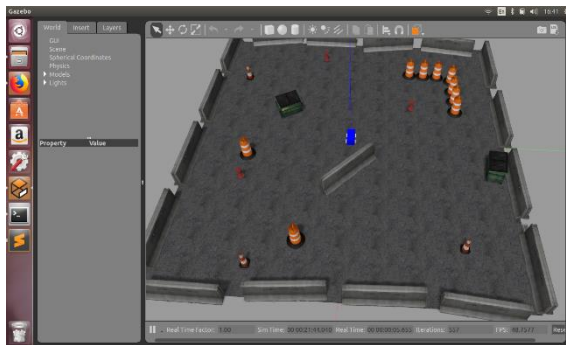
Viết launch file: Để viết một launch file trong ROS, chúng ta cần sử dụng XML để định nghĩa các thông số và lệnh để khởi động các node. Một launch file là một tệp giúp khởi động nhiều node cùng lúc trong ROS.

Chạy ứng dụng: Để chạy một ứng dụng trong ROS, chúng ta cần biên dịch các gói trong workspace và sử dụng lệnh `roslaunch` để khởi động launch file. Sau đó, chúng ta có thể sử dụng các công cụ của ROS để kiểm tra và điều khiển ứng dụng.

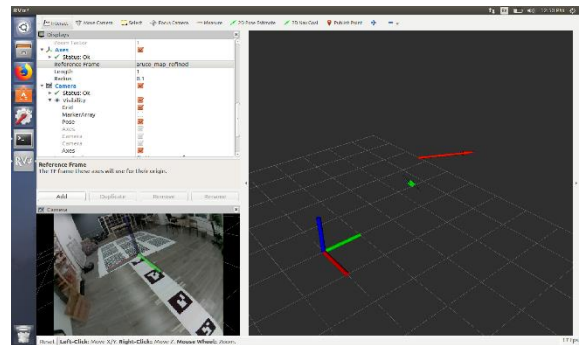
1.4.3. Tại sao lựa chọn ROS

ROS cung cấp cho chúng ta một môi trường phát triển mạnh mẽ và linh hoạt. Với việc hỗ trợ nhiều ngôn ngữ lập trình như C++, Python và Java, người dùng có thể chọn ngôn ngữ phù hợp với nhu cầu và kinh nghiệm của mình. Ngoài ra, ROS cũng đi kèm với các công cụ và thư viện hữu ích để xây dựng các ứng dụng robot phức tạp một cách dễ dàng.

ROS cung cấp một hệ thống giao tiếp tiện lợi cho các module của robot. Bằng cách sử dụng cơ chế truyền tin nhắn, các module có thể giao tiếp và tương tác với nhau một cách dễ dàng. Bên cạnh đó, hệ thống quản lý gói của ROS là một công cụ quan trọng giúp tổ chức và phân phối mã nguồn và tài nguyên liên quan đến robot, giúp người dùng dễ dàng cài đặt, cập nhật và quản lý các gói ROS từ kho lưu trữ chính thức. Điều này giúp giải quyết vấn đề phụ thuộc và tăng khả năng phát triển và triển khai robot.



a. Giao diện phần mềm Gazebo



b. Giao diện phần mềm RViz

Hình 1.7. Giao diện phần mềm Gazebo và phần mềm RViz.

Ngoài ra ROS còn có các công cụ mô phỏng mạnh mẽ như Gazebo (Hình 1.7.a), Rviz (Hình 1.7.b) và Stage, giúp người dùng thử nghiệm và xem trước các ứng dụng robot một cách hiệu quả trước khi triển khai trên robot thực tế. Các công cụ mô phỏng này giúp tiết kiệm thời gian và tài nguyên trong quá trình phát triển và kiểm tra robot.

Cuối cùng, ROS có một cộng đồng lớn và nhiều tài liệu phong phú. Cộng đồng này cung cấp sự hỗ trợ, chia sẻ kiến thức và kinh nghiệm phát triển robot. Tài liệu phong phú và diễn đàn giúp người dùng tìm hiểu và giải quyết các vấn đề trong quá trình lập trình robot.

Có thể thấy, ROS được lựa chọn vì môi trường phát triển mạnh mẽ, hệ thống giao tiếp linh hoạt, quản lý gói tiện lợi, công cụ mô phỏng hiệu quả và cộng đồng hỗ trợ rộng lớn. Sự kết hợp của những yếu tố này làm cho ROS trở thành một lựa chọn hấp dẫn và mạnh mẽ cho lập trình robot.

1.5. Đặt vấn đề nghiên cứu

Trong lĩnh vực robot di động, việc xác định vị trí chính xác và xây dựng bản đồ môi trường là hai yếu tố cơ bản để đảm bảo hoạt động hiệu quả và an toàn của robot. Cải thiện thuật toán tự động xác định vị trí và dựng bản đồ 2D giúp robot hoạt động hiệu quả, an toàn và đáng tin cậy. Đồng thời, việc nghiên cứu và phát triển các phương pháp và thuật toán mới cũng tạo ra tiềm năng ứng dụng trong các lĩnh vực như robot hướng dẫn, dịch vụ hỗ trợ, tự động hóa gia đình và nhiều ứng dụng khác.

Đề tài sẽ triển khai giải quyết bài toán tự động xác định vị trí và xây dựng bản đồ hai chiều cho robot di động mô hình TurtleBot 3 phiên bản Burger hoặc Waffle, thực hiện trên môi trường Ubuntu 20.04 - ROS Noetic. Đề tài thực hiện quá trình mô phỏng hoạt động của robot và hiển thị kết quả bài toán trên phần mềm mô phỏng Gazebo và RViz. Bài toán không hướng tới việc xây dựng lại chương trình tạo bản đồ từ đầu mà thực hiện áp dụng các nền tảng và thư viện được xây dựng sẵn trên môi trường ROS và thực hiện xây dựng phương hướng cải thiện để nâng cao độ chính xác và hiệu quả của các chương trình gốc.

Mục tiêu của đề tài "**Nghiên cứu cải thiện thuật toán tự động xác định vị trí và dựng bản đồ 2D cho mô hình mobile robot trong nhà**" là nghiên cứu và cải thiện thuật toán tự động xác định vị trí và dựng bản đồ 2D cho mô hình mobile robot trong nhà. Cải thiện thuật toán nhằm nâng cao chính xác, hiệu suất và ổn định của quá trình xác định vị trí và xây dựng bản đồ. Áp dụng các phương pháp, kỹ thuật và công nghệ mới nhất trong lĩnh vực này giúp tăng cường khả năng tự động của robot và giảm sự phụ thuộc vào hệ thống thủ công. Việc cải thiện thuật toán trong việc xác định vị trí chính xác và xây dựng bản đồ môi trường 2D cho robot mang lại nhiều lợi ích và ứng dụng trong các lĩnh vực thực tế. Qua quá trình nghiên cứu và phát triển, hy vọng sẽ đóng góp vào sự tiến bộ và phát triển của công nghệ robot di động.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ MÔ HÌNH ROBOT

2.1. Cơ sở lý thuyết

2.1.1. LiDAR SLAM

LiDAR SLAM là phương pháp được nghiên cứu ngay từ khi xuất hiện bài toán SLAM, dựa trên cảm biến quét laser sử dụng chùm tia hồng ngoại để thu được khoảng cách từ robot đến các vật cản trong môi trường. Phương pháp này sử dụng cảm biến laser để thực hiện xác định vị trí và dựng bản đồ đồng thời (SLAM - Simultaneous Localization And Mapping) cho robot di động. LiDAR (Light Detection And Ranging) là một loại cảm biến có khả năng đo khoảng cách đến các đối tượng bằng cách chiếu các xung laser lên chúng và nhận lại tín hiệu phản xạ. Bằng cách quét laser theo nhiều hướng khác nhau, LiDAR có thể tạo ra một bản đồ 3D của môi trường xung quanh robot.

LiDAR SLAM có nhiều ứng dụng trong các lĩnh vực như vận chuyển tự động, cứu hộ, thám hiểm, khảo sát địa hình, v.v. LiDAR SLAM giúp robot có thể di chuyển trong các môi trường không xác định và phức tạp mà không cần sử dụng các điểm mốc hay GPS. LiDAR SLAM cũng giúp robot có thể lập kế hoạch đường đi, tránh chướng ngại vật và giao tiếp với các robot khác.

LiDAR SLAM gồm hai bước chính: front-end và back-end. Front-end là bước xử lý dữ liệu từ cảm biến LiDAR để trích xuất các thông tin về môi trường và vị trí của robot. Front-end có thể sử dụng các thuật toán như ICP (Iterative Closest Point), NDT (Normal Distributions Transform), LOAM (Lidar Odometry And Mapping), v.v. Back-end là bước tối ưu hóa vị trí và bản đồ dựa trên các thông tin từ front-end. Back-end có thể sử dụng các thuật toán như EKF (Extended Kalman Filter), Graph SLAM, Factor Graph, v.v.

Một trong những ứng dụng của LiDAR SLAM là Google Cartographer, một thuật toán SLAM 2D và 3D cho robot di động. Google Cartographer sử dụng dữ liệu từ cảm biến LiDAR để xây dựng bản đồ 2D hoặc 3D của môi trường và xác định vị trí của robot trong bản đồ. Google Cartographer có thể hoạt động trong nhiều loại môi trường khác nhau, từ trong nhà đến ngoài trời.

Tóm lại, LiDAR SLAM là một phương pháp hiệu quả và tiên tiến để giải quyết bài toán xác định vị trí và dựng bản đồ cho robot di động. LiDAR SLAM mang lại nhiều lợi ích cho việc phát triển và ứng dụng của robot trong các lĩnh vực khác nhau.

LiDAR SLAM (Simultaneous Localization and Mapping) là một phương pháp trong lĩnh vực robot di động và thị giác máy tính, được sử dụng để xác định vị trí của robot và đồng thời xây dựng bản đồ của môi trường xung quanh robot bằng sử dụng cảm biến LiDAR (Light Detection and Ranging).

Cảm biến LiDAR hoạt động bằng cách gửi ra một tia laser và đo thời gian mà tia laser phản xạ lại từ các vật thể trong môi trường. Thông qua việc phân tích dữ liệu từ các điểm phản xạ, LiDAR có khả năng xác định khoảng cách và hình dạng của các vật thể xung quanh robot. Điều này cho phép cảm biến LiDAR tạo ra các điểm dữ liệu 3D (điểm đám mây) mô tả môi trường xung quanh.



Hình 2.1. Cảm biến LiDAR.

LiDAR SLAM kết hợp việc sử dụng cảm biến LiDAR và thuật toán SLAM để giải quyết hai vấn đề chính: xác định vị trí của robot trong môi trường chưa biết trước và đồng thời xây dựng bản đồ của môi trường đó. Thông qua quá trình quét và thu thập dữ liệu từ cảm biến LiDAR, thuật toán SLAM sẽ xử lý các điểm dữ liệu 3D và ánh xạ chúng vào một bản đồ 2D hoặc 3D.

Mục tiêu của LiDAR SLAM là cung cấp cho robot khả năng tự định vị chính xác trong môi trường không biết trước và xây dựng bản đồ để tạo ra một hệ thống thông tin môi trường. Qua đó, robot có thể sử dụng thông tin về vị trí và bản đồ để thực hiện các tác vụ như định hướng, lập kế hoạch đường đi, tránh vật cản, hoặc thực hiện các tác vụ

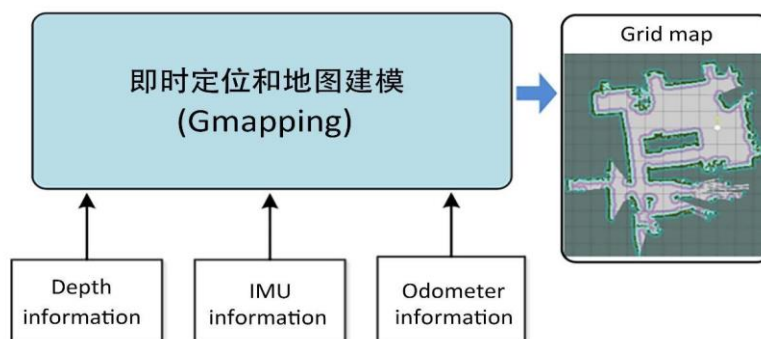
tương tác với môi trường.

Ứng dụng của LiDAR SLAM rất rộng rãi trong các lĩnh vực như robot di động tự hành, xe tự lái, robot dịch vụ, và nhiều ứng dụng khác. Với khả năng xác định vị trí chính xác và xây dựng bản đồ chính xác của môi trường, LiDAR SLAM đóng vai trò quan trọng trong việc nâng cao hiệu suất và an toàn của robot di động trong quá trình hoạt động trong môi trường không biết trước.

2.1.2. Thuật toán Gmapping

Dựa trên ý tưởng cơ bản của LiDAR SLAM, chúng ta có thể đi sâu hơn vào một trong những thuật toán nổi tiếng được sử dụng trong LiDAR SLAM, đó là GMapping.

GMapping là một thuật toán xây dựng bản đồ 2D dựa trên cảm biến LiDAR và được phát triển bởi nhóm GMapping thuộc Willow Garage. GMapping sử dụng phương pháp xác suất Monte Carlo (Monte Carlo localization) để ước lượng vị trí của robot trong môi trường chưa biết trước, cùng với thuật toán giao nhau hiệp phương sai (Iterative Closest Point - ICP) để cập nhật bản đồ mỗi khi robot di chuyển.



Hình 2.2. Cách triển khai thuật toán Gmapping SLAM trong robot di động.

Thuật toán GMapping bắt đầu bằng việc xây dựng một bản đồ rỗng và tiếp tục bằng cách thực hiện quá trình xác định vị trí của robot dựa trên các quét LiDAR. Sử dụng bộ lọc Kalman (Kalman filter), GMapping tạo ra một tập hợp các giả định về vị trí của robot và cập nhật chúng dựa trên dữ liệu LiDAR mới nhận được. Quá trình này được lặp đi lặp lại để cải thiện độ chính xác của vị trí ước lượng.

Đồng thời, GMapping cũng sử dụng thuật toán ICP để so sánh dữ liệu quét LiDAR mới với bản đồ hiện tại và điều chỉnh bản đồ để phù hợp với dữ liệu mới. Quá trình này

giúp cập nhật thông tin về các vật thể và cấu trúc trong môi trường, từ đó xây dựng và cải thiện bản đồ 2D.

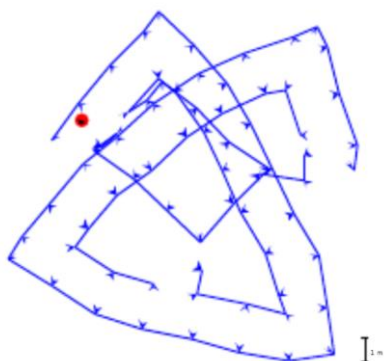
Một trong những ưu điểm của GMapping là khả năng xử lý thời gian thực và khả năng ứng dụng trên robot di động thực tế. GMapping cho phép robot tự định vị và xây dựng bản đồ trong môi trường động và không đảm bảo về sự thay đổi của môi trường.

Với sự linh hoạt và khả năng cải thiện bản đồ, GMapping đã trở thành một trong những lựa chọn phổ biến cho việc triển khai LiDAR SLAM trên robot di động. Nó đã được sử dụng rộng rãi trong các ứng dụng thực tế như xe tự hành, robot dịch vụ và nghiên cứu robot.

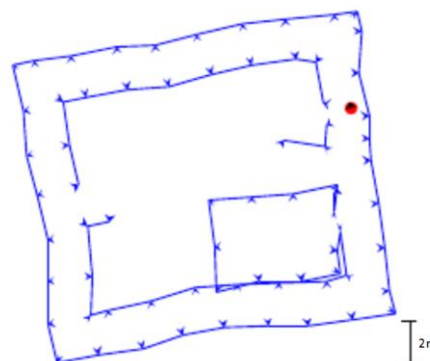
2.1.3. Đóng vòng lặp

Đóng vòng lặp (loop closure) đề cập đến một khái niệm quan trọng trong các hệ thống SLAM (Simultaneous Localization and Mapping), robot cần xác định lại vị trí hiện tại của mình trong bản đồ đã xây dựng để điều chỉnh và cải thiện độ chính xác của vị trí và bản đồ.

Trong quá trình di chuyển, robot thu thập dữ liệu từ các cảm biến như LiDAR, camera hoặc vị trí GPS để tạo bản đồ và ước lượng vị trí. Tuy nhiên, do sai số và nhiễu trong dữ liệu cảm biến, có thể xảy ra hiện tượng mất mát, dịch chuyển hoặc méo mó các khu vực của bản đồ. Đóng vòng lặp là quá kiểm tra khung hình hiện tại với khung hình trước đó, bước khép kín này giúp giảm sai số tích lũy, từ đó tăng độ hiệu quả của thuật toán.



a. Bản đồ khi chưa đóng vòng lặp



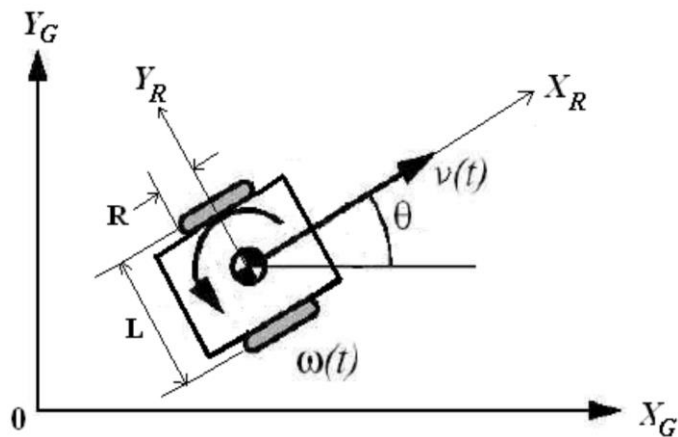
b. Bản đồ khi đóng vòng lặp

Hình 2.3. Minh họa so sánh giữa bản đồ khi có/không sử dụng đóng vòng lặp.

Khi robot quay trở lại một vị trí đã từng đi qua trước đó, thông qua các phép so sánh và phân tích dữ liệu cảm biến, hệ thống SLAM có thể nhận ra rằng robot đã quay lại vị trí đã biết trong bản đồ. Quá trình này được gọi là đóng vòng lặp và giúp cải thiện độ chính xác của bản đồ và ước lượng vị trí.

2.2. Mô hình robot hai bánh vi sai

Mô hình robot di động hai bánh vi sai là một mô hình động học sử dụng để mô phỏng và điều khiển robot di chuyển trên mặt phẳng. Trong mô hình này, robot được xem như một hệ thống đa khối, với hai bánh xe chủ động và khả năng quay độc lập. Mỗi bánh xe có thể di chuyển với một tốc độ riêng, tạo ra hiệu ứng bánh vi sai.



Hình 2.4. Mô hình robot di động hai bánh vi sai.

trong đó:

(X_G, Y_G) : Biểu diễn hệ tọa độ toàn cục.

(X_R, Y_R) : Biểu diễn hệ tọa độ cục bộ gắn liền với robot.

R : Bán kính bánh xe.

L : Khoảng cách giữa hai bánh.

Phương trình động học của robot được mô tả như sau:

$$\begin{cases} \dot{x} = v \cos \theta \\ \dot{y} = v \sin \theta \\ \dot{\theta} = \omega \end{cases}$$

với: (x, y): Tọa độ của robot.

θ : Hướng của robot.

v: vận tốc dài của robot

ω : vận tốc góc của robot.

$$\begin{cases} v = \frac{R(\omega_L + \omega_R)}{2} \\ \omega = \frac{R(\omega_R - \omega_L)}{L} \end{cases}$$

với R là bán kính bánh xe

L là khoảng cách giữa hai bánh

ω_L và ω_R lần lượt là vận tốc góc của bánh trái và bánh phải.

Mô hình động học rời rạc có dạng như sau:

$$\begin{cases} x_{i+1} = x_i + \Delta t.v_i \cos \theta_i \\ y_{i+1} = y_i + \Delta t.v_i \sin \theta_i \\ \theta_{i+1} = \theta_i + \Delta t.\omega_i \end{cases}$$

CHƯƠNG 3: XÂY DỰNG CHƯƠNG TRÌNH

3.1. Mục tiêu đạt được

Mục tiêu cơ bản nhất của đề tài là xây dựng được chương trình điều khiển robot thực hiện các tác vụ sau: tự động định vị và điều hướng di chuyển trong môi trường trong nhà. Trong quá trình di chuyển này, dựa vào tín hiệu thu nhận từ cảm biến LiDar để robot xác định môi trường xung quanh có vật cản hay không, và di chuyển sao cho robot không va vào bất kì vật cản trong phòng. Đồng thời, robot xử lý các tín hiệu thu được từ cảm biến và tính khoảng cách từ nó đến vật cản xung quanh, vừa dựa vào đây và giá trị odometry và ghi lại thành các tập dữ liệu và trực quan hóa thành bản đồ môi trường xung quanh thông qua Gmapping. Cảm biến chính được sử dụng ở đây là LiDar 360 độ để đo đạc và theo dõi môi trường robot đang hoạt động dựa vào những đặc điểm của môi trường là vật cản, giới hạn miền và các phần tử khác. Từ tập dữ liệu thu được ở cảm biến, ta truyền qua bộ lọc trong package Gmapping để tính toán ước lượng một khoảng cách với nhiều lần đo để ước lượng, định tính và dựng lại bản đồ môi trường.

3.2. Đề xuất phương án thực hiện

Phương án thực hiện bài toán từ vấn đề cơ bản nhất sẽ gồm ba thành phần sau:

3.2.1. Mô phỏng động lực học cho robot.

Chương trình mô phỏng trong môi trường Gazebo cho phép xem xét hoạt động của robot trong một môi trường ảo. Bằng cách import mô hình URDF của robot, chúng ta có thể gắn các trục tọa độ và các khớp của robot. Chương trình cũng cho phép mô phỏng các tác động của động lực học robot khi di chuyển trong một môi trường được xây dựng sẵn. Bằng cách tạo các tệp .world và import các mô hình môi trường vào, chúng ta có thể tạo ra các môi trường hoạt động để thử nghiệm robot.

3.2.2. Dựng bản đồ môi trường.

Chương trình dựng bản đồ môi trường cho robot được hiển thị trên RViz, sử dụng các bộ hàm xây dựng trong Gmapping. Quá trình này bắt đầu bằng việc thu thập dữ liệu từ cảm biến khoảng cách LiDAR và tính toán giá trị từ robot đến các vật cản trong môi trường xung quanh. Sau đó, các giá trị này được lưu trữ trong một bộ dữ liệu và truyền

vào bộ lọc Kalman hoặc RBDF để lọc các giá trị nhiễu của cảm biến. Cuối cùng sẽ tính toán khoảng cách trung bình tới một điểm nhất định dựa trên các dữ liệu thu thập được.

Khi robot di chuyển xung quanh để khám phá môi trường, nó thực hiện nhiều lần di chuyển qua cùng một vị trí và thu thập nhiều mẫu dữ liệu. Việc này giúp chương trình tính toán đường biên tới các vật cản chính xác hơn và cập nhật bản đồ môi trường để loại bỏ những sai số trước đó.

Trong quá trình xây dựng bản đồ, Gmapping sử dụng phương pháp SLAM (Simultaneous Localization and Mapping) để đồng thời xác định vị trí của robot và xây dựng bản đồ môi trường xung quanh. Thuật toán SLAM trong Gmapping kết hợp thông tin từ cảm biến LiDAR và dữ liệu về vị trí của robot để tạo ra một bản đồ chi tiết và chính xác.

Kết quả của quá trình dựng bản đồ được hiển thị trực tiếp trên RViz, một công cụ mô phỏng và hiển thị trực quan trong ROS. Trên RViz, ta có thể quan sát bản đồ môi trường mà robot đã khám phá và cập nhật liên tục. Bản đồ này có thể được sử dụng để điều hướng robot trong môi trường, tránh các vật cản và thực hiện các nhiệm vụ khác nhau.

3.2.3. Định vị và điều hướng robot khám phá môi trường.

Robot thực hiện việc định vị và điều hướng tự động trong môi trường chưa được khám phá. Quá trình này bao gồm việc thu thập dữ liệu từ các cảm biến và sử dụng các thuật toán xử lý để xác định vị trí của robot trong không gian và tính toán quỹ đạo di chuyển.

Quá trình định vị và điều hướng của robot diễn ra đồng thời với việc thu bản đồ môi trường. Robot khám phá môi trường bằng cách di chuyển và thu thập dữ liệu từ các cảm biến. Dữ liệu này được sử dụng để cập nhật bản đồ môi trường, ghi lại thông tin về các vật cản, địa hình và các đặc điểm khác trong môi trường.

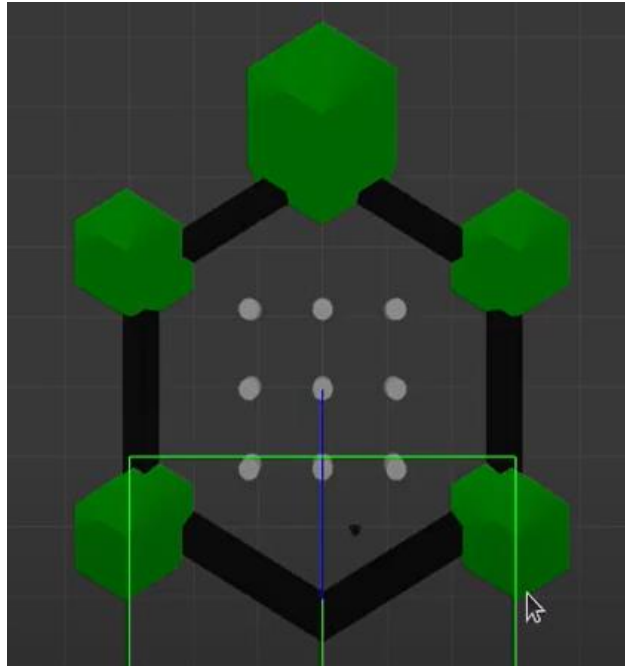
Kết quả của quá trình này là robot có khả năng tự định vị và điều hướng trong môi trường chưa được khám phá. Tín hiệu điều khiển quỹ đạo di chuyển được truyền vào mô hình robot mô phỏng Gazebo, cho phép kiểm tra và đánh giá hiệu suất của robot

trong môi trường ảo.

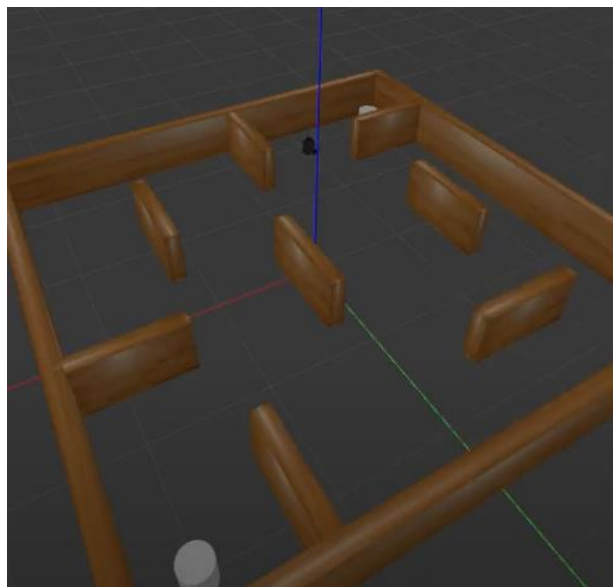
3.3. Triển khai bài toán

3.3.1. Dựng mô hình mô phỏng robot.

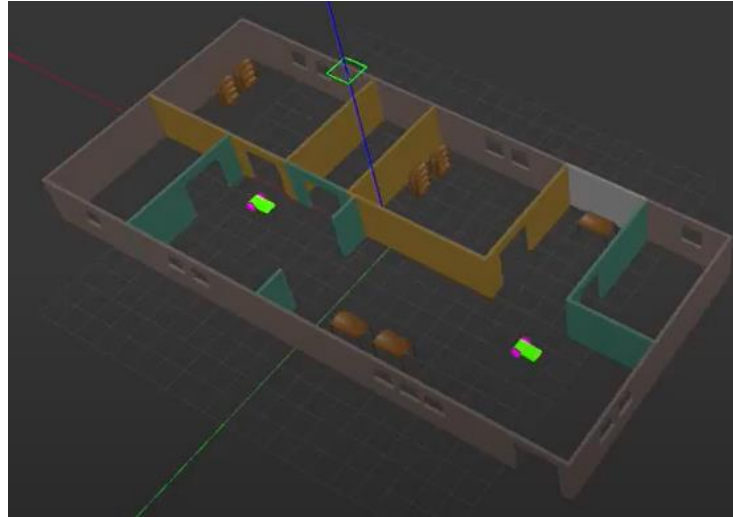
Tạo môi trường cho robot: ba môi trường theo ba mức độ từ dễ đến khó để thực hiện kiểm tra, khảo sát kết quả.



Hình 3.1. Khởi chạy môi trường thứ nhất TB3_Hexa.world.

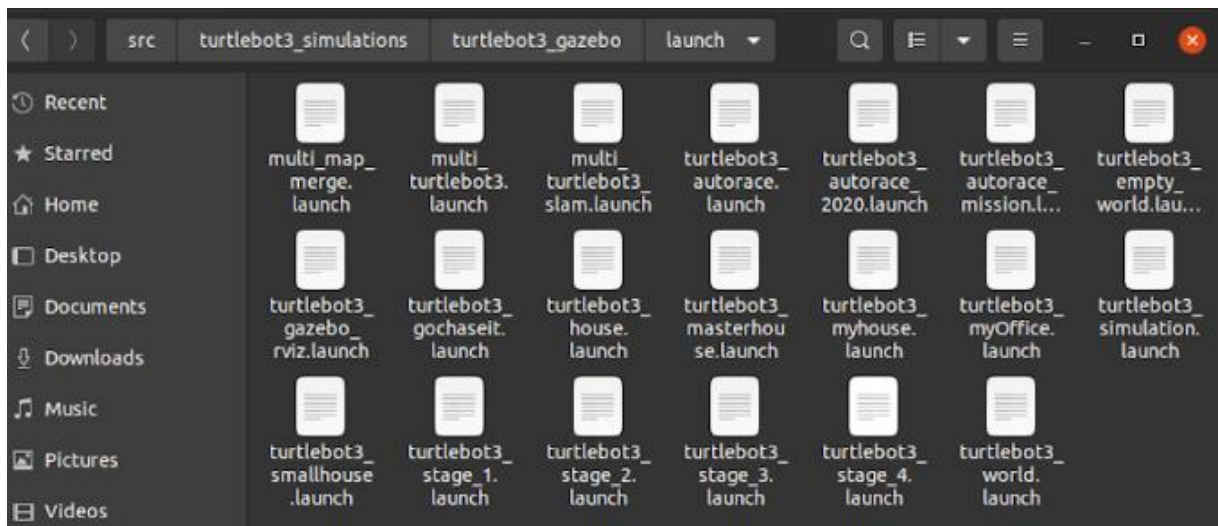


Hình 3.2. Khởi chạy môi trường thứ hai maze.world.



Hình 3.3. Khởi chạy môi trường thứ ba myhouse.world.

Dưới đây là tổng hợp các file world đã tạo làm test cases để chạy các chương trình SLAM ở các môi trường khác nhau.



Hình 3.4. Các file .launch đã tạo tùy chỉnh cho từng môi trường.

3.3.2. Thu bản đồ từ Gmapping:

Bản đồ được phát triển từ dữ liệu đầu tiên thu được ở môi trường sau khi chạy mô hình turtlebot trong Gazebo và chương trình Slam. Mô hình robot gồm hai bánh xe vi sai và một cảm biến ngoại vi phát hiện chuyển động gắn ở khung xe. Quá trình thu bản đồ bằng thuật toán gmapping sử dụng dữ liệu odometry thu được từ cảm biến và LiDar gắn thẳng, cố định, chung gốc với gốc tọa độ của robot, tầm chiếu tia laser song song

với nền, ghi lại mặt cắt của môi trường tại chiều cao chiếu.

Chương trình sử dụng kết hợp giữa số liệu đo khoảng cách chiếu từ tia laser của LiDar đến vật cản đồng thời với ảnh chiều sâu quy đổi sang dạng 2D thu được ở cảm biến ngoại vi trên. Phục vụ trường hợp có vật cản động. Cảm biến ngoại vi chụp lại ảnh môi trường có chiều sâu bằng cảm biến hồng ngoại, dữ liệu thu được là dữ liệu hồng ngoại 16 bits với độ phân giải 640 x 480 trong định dạng ảnh. Bộ dữ liệu này được thu thập và dựng thành đám mây quét chiều sâu 3D xung quanh robot. Sau đó bộ này được quy đổi và bổ sung vào dựng bản đồ từ từ quét laser 2D phía trên. Khi cấp đủ cho chương trình slam gmapping các tham số cần thiết, chương trình sẽ cho ra một bản đồ dựng ở phần mềm RViz. Bản đồ được lưu lại khi hoàn chỉnh bằng server /map_server, node map_saver.

3.3.3. Truyền quỹ đạo vào mô hình robot

Quỹ đạo được truyền vào robot từ mức đơn giản nhất – quỹ đạo tránh vật cản thông thường, không có điều kiện lặp, ngắt và quỹ đạo cải tiến hơn, có điều kiện di chuyển, có định hướng cách di chuyển khám phá bản đồ và điều kiện thoát vòng lặp khi đã thu bản đồ đủ hoặc không thể tiếp tục thu bản đồ.

Hai quỹ đạo trên được xây dựng và thử cho ba mô hình môi trường khác nhau đã được xây dựng để khảo sát hoạt động, hành vi của robot trong các môi trường này và xem xét mức độ, khả năng đáp ứng của thuật toán có phù hợp không, từ đó tiếp tục đưa ra hướng phát triển để cải thiện chương trình hơn nữa.

3.4. Quá trình chạy chương trình.

3.4.1. Khởi tạo môi trường làm việc

```
# Cài đặt ROS Noetic trên Ubuntu nếu chưa có theo tutorial trên ROS Wiki  
# tại: http://wiki.ros.org/noetic/Installation/Ubuntu  
  
# Cài đặt một số package cần thiết của ROS cho quá trình chạy, mở terminal  
và chạy các dòng lệnh dưới đây:  
$ sudo apt-get install ros-noetic-joy ros-noetic-teleop-twist-joy
```

```

$ sudo apt-get install ros-noetic-teleop-twist-keyboard ros-noetic-
laser-proc

$ sudo apt-get install ros-noetic-rgbd-launch ros-noetic-depthimage-to-
laserscan

$ sudo apt-get install ros-noetic-rosserial-arduino ros-noetic-
rosserial-python

$ sudo apt-get install ros-noetic-rosserial-server ros-noetic-rosserial-
client

$ sudo apt-get install ros-noetic-rosserial-msgs ros-noetic-amcl ros-
noetic-map-server

$ sudo apt-get install ros-noetic-move-base ros-noetic-urdf ros-noetic-
xacro

$ sudo apt-get install ros-noetic-compressed-image-transport ros-noetic-
rqt

$ sudo apt-get install ros-noetic-gmapping ros-noetic-navigation ros-
noetic-interactive-markers

$ sudo apt install ros-noetic-base-local-planner

$ sudo apt install ros-noetic-navfn

$ sudo apt-get install ros-noetic-gazebo-ros

# Cập nhật lại hệ thống:

$ sudo apt-get update

$ sudo apt-get upgrade

# Tạo workspace làm việc: Mở một terminal mới về địa chỉ gốc Home.
# Tạo một địa chỉ mới đến thư mục workspace.

$ mkdir ~/SlamProject

# Tạo một folder src bên trong chứa source code

$ mkdir src

$ catkin init      # -> kết quả true: có thể tạo workspace

$ catkin build     # xây dựng lại môi trường làm việc

# Cài đặt package gốc của model robot TurtleBot3

$ cd ~/SlamProject/src

$ git clone https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git

```

```

$ git clone https://github.com/ROBOTIS-GIT/turtlebot3.git
$ git clone https://github.com/ROBOTIS-GIT/turtlebot3_simulations.git
$ sudo apt install ros-noetic-turtle3-gazebo
$ cd ..
$ catkin build

# Cần xây dựng lại thư mục workspace sau mỗi lần thay đổi
# nội dung/package bên trong để có thể chạy được chương trình.

# Viết lại file source
$ gedit ~/.bashrc

# Bên trong file source .bashrc, thêm những dòng sau vào vào lưu lại.
alias burger='export TURTLEBOT3_MODEL=burger'
alias waffle='export TURTLEBOT3_MODEL=waffle'
alias tb3fake='roslaunch turtlebot3_fake turtlebot3_fake.launch'
alias tb3teleop='roslaunch turtlebot3_teleop
turtlebot3_teleop_key.launch'
alias tb3='roslaunch turtlebot3_gazebo turtlebot3_empty_world.launch'
alias tb3maze='roslaunch turtlebot3_gazebo turtlebot3_world.launch'
alias tb3house='roslaunch turtlebot3_gazebo turtlebot3_house.launch'

source /opt/ros/noetic/setup.bash
source /home/ryuzuu/SlamProject/devel/setup.bash
export TURTLEBOT3_MODEL=waffle
export TURTLEBOT3_MODEL=burger
export SVGA_VGPU10=0
$ cd ..
$ catkin build

```

3.4.2. Chạy chương trình mô phỏng cho robot.

```

# Để đề phòng không chạy được, khi mở terminal mới chạy file .launch
# cần thêm các câu lệnh gọi source và khai báo model robot sử dụng.

```

```

# Gọi source cho môi trường làm việc
$ . ~/SlamProject/devel/setup.bash

# Khai báo model sử dụng, chỉ cần gọi 1 trong 2 dòng.
$ export TURTLEBOT3_MODEL=waffle
$ export TURTLEBOT3_MODEL=burger

# Chạy 1 trong 3 chương trình mô phỏng cho turtlebot3 trên gazebo

# môi trường 1: mức độ dễ
$ roslaunch turtlebot3_gazebo turtlebot3_world.launch

# môi trường 2: mức độ trung bình
$ roslaunch turtlebot3_gazebo turtlebot3_maze.launch

# môi trường 3: mức độ cao
$ roslaunch turtlebot3_gazebo turtlebot3_myhouse.launch

# Chương trình mô phỏng động học động lực học cho mô hình robot
# có thể thay thế tên file .launch bằng một trong những file launch
# đã được xây dựng ở trên để thay đổi môi trường khác và chạy với
# thông số được tinh chỉnh khác trên model robot.

# Mở một terminal mới, lặp lại bước source và export model nếu có lỗi
# Chạy chương trình ghi bản đồ bằng Gmapping
$ roslaunch turtlebot3_slam turtlebot3_slam.launch
slam_methods:=gmapping

# chương trình gmapping hiển thị kết quả trên rviz -
# khi này robot lấy bản đồ ở trạng thái tĩnh không chuyển động

```

```

# Mở một terminal mới, chạy chương trình điều hướng cho robot
# chạy trong môi trường và tránh vật cản, bản đồ thu được ở rviz.
$ roslaunch turtlebot3_gazebo turtlebot3_simulation.launch

# khi được truyền quỹ đạo và vận tốc, robot bắt đầu di chuyển
# vòng quanh bản đồ để thu tín hiệu của môi trường, mở rộng bản đồ.

# Mở một terminal khác, chạy dòng code sau để lưu lại bản đồ.
$ rosrun map_server map_saver -f ~/map

```

3.4.3. Một số thao tác lấy dữ liệu trong quá trình thực hiện.

```

# Bắt đầu chương trình tới node ros master để truy cập các node con.
$ roscore

catkin_make -DCATKIN_WHITELIST_PACKAGES="ros_essentials_cpp" -
DPYTHON_EXECUTABLE=/usr/bin/python3
$ catkin_make -DPYTHON_EXECUTABLE=/usr/bin/python3

# Kiểm tra các node đang hoạt động
$ rostopic list

# Truy cập tới địa chỉ folder chứa node nào đó để có thể launch node
# và tìm thêm thông tin về node này. Cú pháp thực hiện như sau:
$ roscd <node_name>

# Chạy các ros packages với cú pháp sau:
# rosrun <package_name> <node_name>
$ rosrun ros_essentials_cpp talker_node
$ rosrun ros_essentials_cpp listener_node
$ rosrun ros_essentials_cpp talker.py

```

```
$ rosrun ros_essentials_cpp listener.py

# Lấy thông tin về một node nào đó:
$ rosnode info /turtlesim

# Lấy thông tin về topics:
$ rostopic info /turtle1/cmd_vel

# Lấy thông tin về vận tốc của robot trong thời gian thực
$ rostopic echo /turtle1/cmd_vel

# Hiển thị toàn bộ topics đang hoạt động:
$ rostopic list

# Hiển thị toàn bộ node/process đang hoạt động:
$ rosnode list

# Launch gazebo
# roslaunch <package_name> <launch_name>
$ roslaunch gazebo_ros empty_world.launch

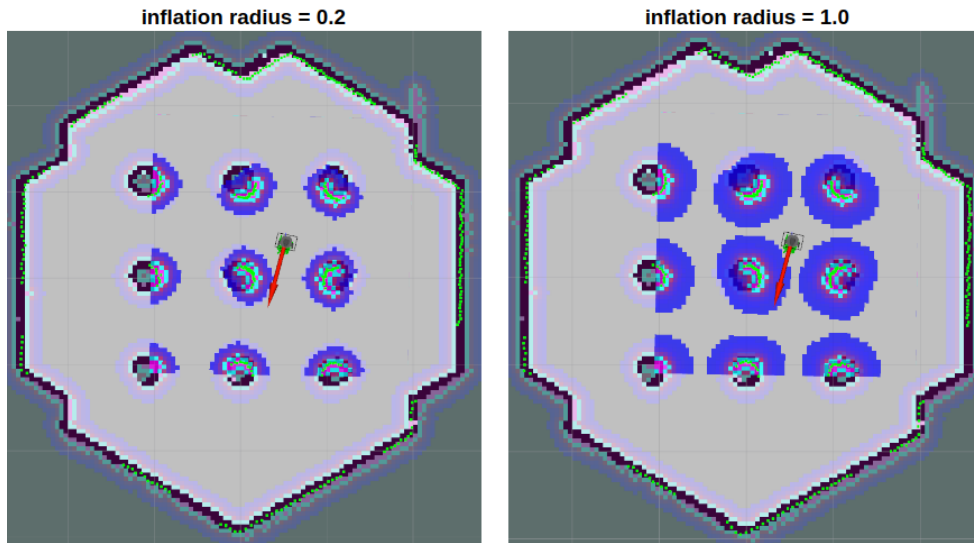
# Launch rviz
$ rosrun rviz rviz
$ rosrun rqt_plot rqt_plot
```

3.5. Tiêu chí cải thiện mô hình.

3.5.1. Inflation_radius

Inflation_radius hay còn gọi là bán kính tiệm cận với vật cản. Thông số này xác định vùng bao quanh vật cản mà robot đo được, đây là một vùng robot sẽ không di chuyển tới để tránh đâm vào vật cản, khoảng cách này nên lớn hơn bán kính của robot, khi chỉnh thông số này thấp xuống sẽ thu hẹp khoảng tiếp cận của robot đến vật cản, phục vụ việc đưa robot tiếp cận phạm vi hẹp. Thay đổi tham số tại file dưới đây:

turtlebot3_navigation/param/costmap_common_param_\${TB3_MODEL}.yaml



Hình 3.5. Phạm vi tiếp cận của robot sau và trước khi điều chỉnh *inflation_radius*.

3.5.2. Thay đổi tham số đầu vào.

Thay đổi vận tốc chạy của robot đến vận tốc giới hạn cho phép của cả vận tốc góc và vận tốc dài, cho phép robot di chuyển nhanh hơn trong bản đồ rộng, thực hiện quá trình khám phá môi trường trong quãng thời gian ngắn hơn. Điều chỉnh ở file: *turtlebot3_navigation/param/dwa_local_planner_params_\${TB3_MODEL}.yaml*

Tăng phạm vi quét của LiDar đến giới hạn lớn nhất cho phép, tiết kiệm thời gian khảo sát môi trường xung quanh robot với phạm vi rộng. Điều chỉnh ở file *turtlebot3_description/urdf/turtlebot3_waffle.gazebo.xacro*

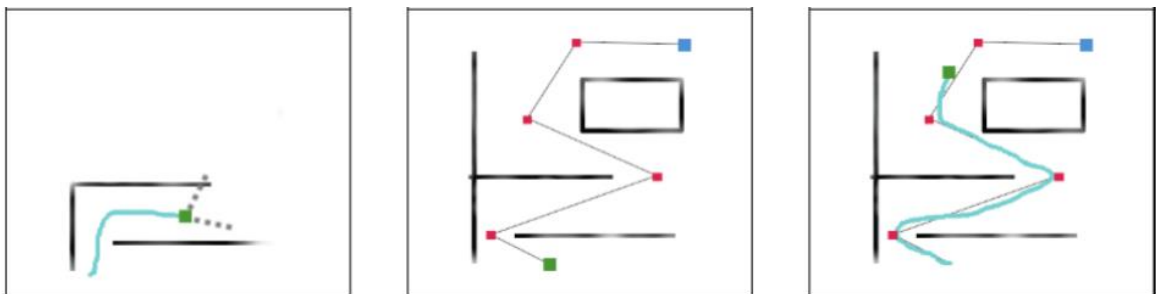
Tạo điều kiện bắt buộc ngắt cho chương trình khi robot va chạm và không thể đi tiếp, khi robot đi ra ngoài phạm vi lập bản đồ, sau một khoảng thời gian lặp lâu mà không vẽ thêm được phần bản đồ mới. Đổi điều kiện lặp trong chương trình, giảm số lần đo đạc và giảm số lần đếm cho phép robot đi vòng quanh một khu vực.

3.5.3. Tối ưu hóa quãng đường thực hiện.

Thực hiện chương trình tự động khám phá môi trường với thuật toán tìm kiếm liên tục thực hiện với đầu vào là các điểm đánh dấu trên phần mềm Rviz về khu vực môi trường cần đi tới, thực hiện khám phá hết các khu vực này trên bản đồ để đáp ứng được

đầu bài dựng bản đồ hoàn chỉnh. Robot thực hiện việc di chuyển tránh vật cản và bám theo mép tường để đi.

Robot được nhận điểm vị trí nó cần đi đến dựa vào phần chưa được khám phá trên bản đồ, từ đây robot thực hiện tính toán và khởi tạo ra điểm phía trước nó để đi đến mục tiêu ngắn nhất. Điểm này cần thỏa mãn phục vụ robot di chuyển quãng đường ngắn nhất tới mục tiêu và không nằm trên đoạn đường gấp chướng ngại vật. Robot liên tục tính toán dựa trên tập dữ liệu đo được ở LiDar để khởi tạo ra điểm thuộc quỹ đạo di chuyển của nó và di chuyển theo các điểm tạo ra để đi tới điểm mục tiêu.



Hình 3.6. Quá trình tạo quãng đường tối ưu tự động của robot.

CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ

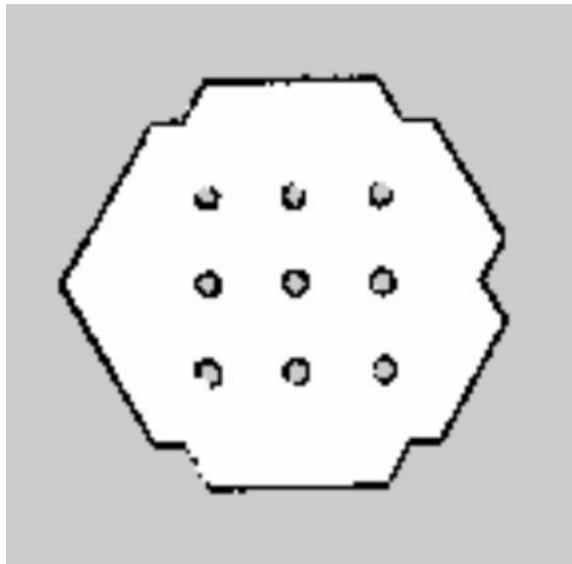
4.1. Testcase 1: Môi trường đơn giản.

4.1.1. Mức tiếp cận cơ bản.

Mô tả đầu vào của bài toán: thực hiện xây dựng bản đồ cho môi trường TB3.world. Chương trình thực hiện vẽ bản đồ với môi trường nhỏ, ít vật cản, khoảng cách giữa các vật cản rộng, có tính đối xứng. Hàm dựng bản đồ chạy song song với hàm di chuyển tránh vật cản thông thường, không có điều kiện lặp cũng không có điều kiện di chuyển.

Kết quả mô phỏng: Video kết quả mô phỏng được thực hiện [tại đây](#).

Các file kết quả thu được về hình ảnh, video quá trình và bộ dữ liệu thu được lập bản đồ được đặt ở [thư mục sau](#). Hình ảnh dưới đây là bản đồ robot thu được sau quá trình thực hiện:



Hình 4.1. Bản đồ môi trường TB3.world.

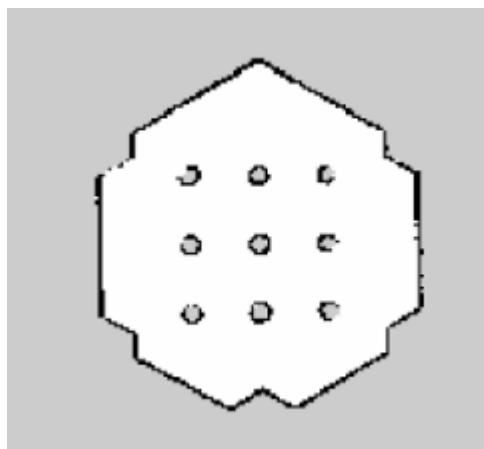
Nhận xét kết quả thu được: Chương trình cơ bản đủ đáp ứng cho điều kiện của bài toán với môi trường mức độ dễ. Bản đồ thu được có độ chính xác cao nhưng thời gian thực hiện ghi lại bản đồ lâu, robot mất nhiều thời gian đi lặp lại nhiều lần tại một địa điểm đã biết mà không có điều kiện phá loop để đi tới vị trí chưa được khám phá. Quãng đường di chuyển của robot chưa được tối ưu, còn lặp lại nhiều lần, còn bị dính vào vòng lặp vô hạn, cần phải ngừng chương trình và lưu bản đồ thủ công.

4.1.2. Cải thiện chương trình.

Mô tả sự cải thiện: Thêm vào chương trình quỹ đạo di chuyển của robot một điều kiện di chuyển theo hình xoắn ốc từ chính giữa môi trường và đi vòng ra xung quanh, trong quá trình đồng thời thực hiện tránh vật cản, chương trình ngắt sau khi robot đi quanh bản đồ bốn lần.

Kết quả mô phỏng: Video kết quả mô phỏng thực hiện được trình bày [tại đây](#).

Các file kết quả thu được về hình ảnh, video quá trình và bộ dữ liệu thu được lập bản đồ được đặt ở [thư mục sau](#). Hình ảnh dưới đây là bản đồ robot thu được sau quá trình thực hiện:



Hình 4.2 Bản đồ môi trường TB3.world với chương trình cải thiện.

Nhận xét kết quả thu được: Bản đồ thu được có độ chính xác cao, thời gian thực hiện ghi lại bản đồ được rút ngắn hơn nhiều so với lần thử trước. Robot không mất nhiều thời gian đi lặp lại nhiều lần tại một địa điểm đã biết mà có thể di chuyển vòng quanh đi tới vị trí chưa được khám phá. Quỹ đường di chuyển của robot được tối ưu hơn trước, giảm tình trạng lặp nhiều lần, có thể tự ngắt chương trình và lưu bản đồ thủ công.

4.2. Testcase 2: Môi trường bản đồ mê cung.

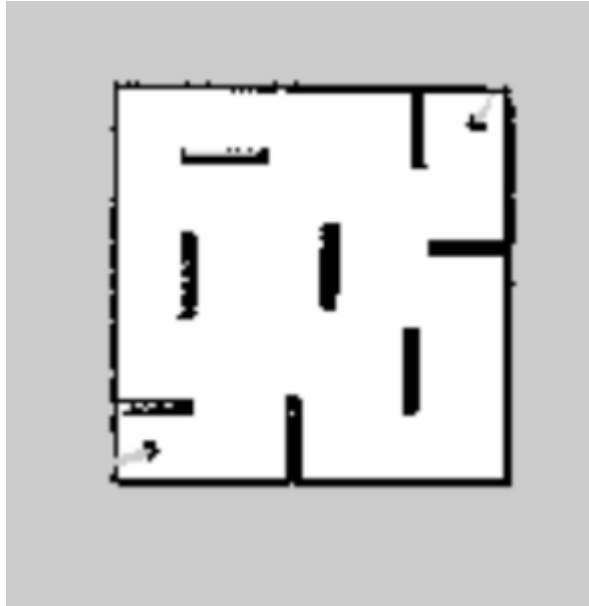
4.2.1. Mức tiếp cận cơ bản.

Mô tả đầu vào của bài toán: thực hiện xây dựng bản đồ cho môi trường maze.world. Chương trình thực hiện vẽ bản đồ với môi trường cỡ vừa, nhiều vật cản, khoảng cách giữa các vật cản hẹp, có tính đối xứng. Hàm dựng bản đồ chạy song song với hàm di chuyển tránh vật cản thông thường, không có điều kiện lặp cũng không có điều kiện di

chuyên.

Kết quả mô phỏng: Video kết quả mô phỏng được thực hiện [tại đây](#).

Các file kết quả thu được về hình ảnh, video quá trình và bộ dữ liệu thu được lập bản đồ được đặt ở [thư mục sau](#). Hình ảnh dưới đây là bản đồ robot thu được sau quá trình thực hiện:



Hình 4.3. Bản đồ môi trường *maze.world*.

Nhận xét kết quả thu được: Quá trình di chuyển của robot trong mê cung chậm bởi robot tốn nhiều thời gian đo khi gặp vật cản là các bức chắn, vách ngăn trong mê cung, thời gian khảo sát lớn. Phạm vi di chuyển đặt cách vật cản còn lớn, vì vậy với con đường chắn hai bên cạnh, robot không đi vào đây dù kích thước đường đủ lớn vừa robot, gây ra tình trạng thiếu một số phần của bản đồ. Robot vẫn còn tình trạng lặp vô hạn, không thể đi đến một số vị trí ở góc bản đồ, cũng như khoảng xét cách vật cảnh tránh còn quá xa, khiến quá trình tính toán chậm, di chuyển không ổn định, bị giật.

Chương trình đặt điều kiện đủ khả năng tạo dựng bản đồ cho môi trường với độ phức tạp mức trung, tuy nhiên hành vi của robot vẫn còn nhiều phần cần cải thiện,

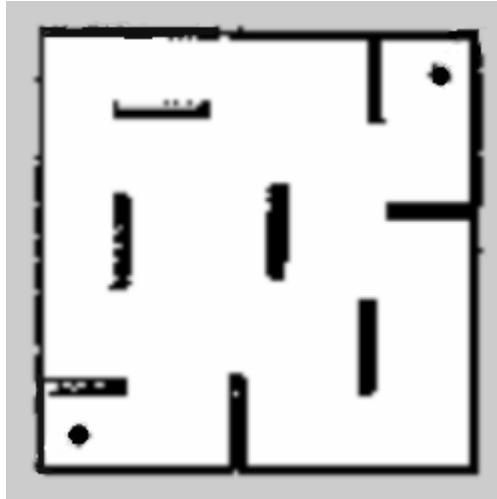
4.2.2. Cải thiện chương trình.

Mô tả sự cải thiện trong chương trình: robot có thể di chuyển trong phạm vi vật cản xếp hẹp hơn sau khi giảm độ tiệm cận với vật cản, robot sau khi di chuyển vòng quanh bản đồ với quỹ đạo từ chương trình giải mê cung - có kết hợp giữa tránh vật cản,

bám mép tường và di chuyển quét phần chưa biết của bản đồ.

Kết quả mô phỏng: Video kết quả mô phỏng được thực hiện [tại đây](#).

Các file kết quả thu được về hình ảnh, video quá trình và bộ dữ liệu thu được lập bản đồ được đặt ở [thư mục sau](#). Hình ảnh dưới đây là bản đồ robot thu được sau quá trình thực hiện:



Hình 4.4. Bản đồ môi trường maze.world.

Nhận xét kết quả thu được: Quá trình di chuyển của robot trong mê cung chậm bởi robot tốn nhiều thời gian đo khi gặp vật cản là các bức chắn, vách ngăn trong mê cung, thời gian khảo sát lớn. Phạm vi di chuyển đặt cách vật cản còn lớn, vì vậy với con đường chắn hai bên cạnh, robot không đi vào đây dù kích thước đường đủ lớn vừa robot, gây ra tình trạng thiếu một số phần của bản đồ. Robot vẫn còn tình trạng lặp vô hạn, không thể đi đến một số vị trí ở góc bản đồ, cũng như khoảng xét cách vật cảnh tránh còn quá xa, khiến quá trình tính toán chậm, di chuyển không ổn định, bị giật.

Chương trình đặt điều kiện đủ khả năng tạo dựng bản đồ cho môi trường với độ phức tạp mức trung, tuy nhiên hành vi của robot vẫn còn nhiều phần cần cải thiện,

4.3. Testcase 3: Môi trường bản đồ trong phòng.

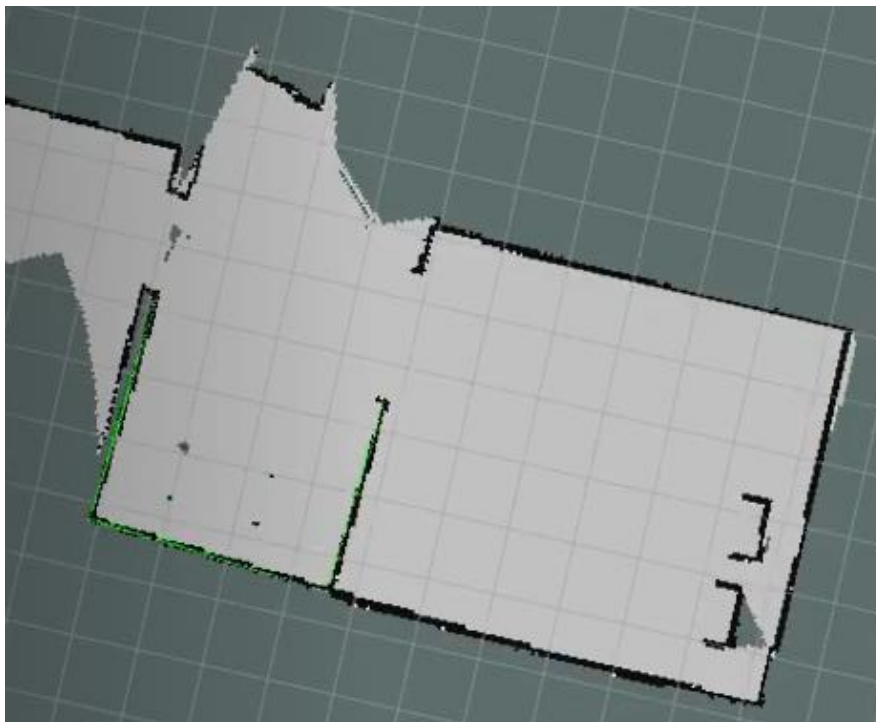
4.3.1. Mức tiếp cận cơ bản.

Mô tả đầu vào của bài toán: thực hiện xây dựng bản đồ cho môi trường myhouse.world. Chương trình thực hiện vẽ bản đồ với môi trường cỡ lớn, số lượng vật cản ít nhưng chi tiết nhỏ, khó thực hiện đo lường; nhiều vách ngăn, tường chắn, môi

trường trong nhà bố trí nhiều phòng, phân gian. Mục tiêu của bài là thu được bản đồ hoàn chỉnh cho toàn bộ các phòng. Robot biết vào phòng khi tiến hành thực hiện khảo sát môi trường chưa rõ và ra ngoài khi đã thu xong bản đồ.

Kết quả mô phỏng: Video kết quả mô phỏng được thực hiện [tại đây](#).

Các file kết quả thu được về hình ảnh, video quá trình và bộ dữ liệu thu được lập bản đồ được đặt ở [thư mục này](#) và [thư mục này](#). Hình ảnh dưới đây là bản đồ robot thu được sau quá trình thực hiện:



Hình 4.5. Bản đồ môi trường myhouse.world.

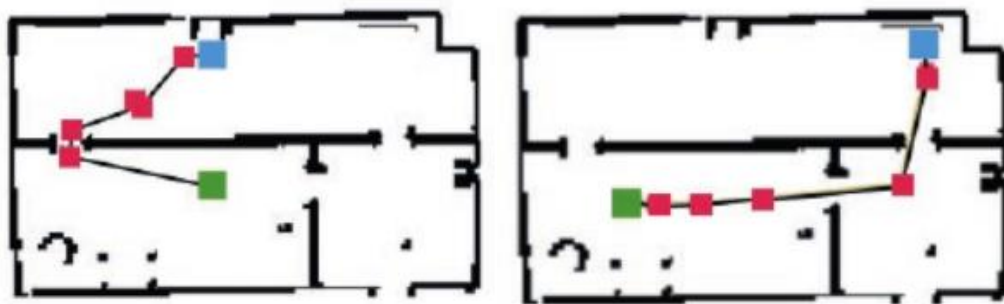
Nhận xét kết quả thu được: Chương trình chưa đáp ứng được điều kiện đủ để giải quyết bài toán. Robot di chuyển ngẫu nhiên theo phân bố xác suất nên hầu như đều có thể khảo sát được hết căn phòng đầu tiên đặt mô hình. Tuy nhiên, việc lặp lại quãng đường còn nhiều, robot chưa có điều kiện thoát để đánh giá xem nó đã thực hiện vẽ xong bản đồ phòng đó rồi và cần ra khỏi phòng hay chưa. Đường đi của robot chưa được tối ưu, quãng đường đã thực hiện ít sự tính toán đặt trước, còn lặp lại nhiều lần và không theo mong muốn đưa ra ban đầu.

Có xảy ra tình trạng robot không thể đi hết bản đồ mà đã thoát ra vì va vào vật cản nhỏ, không thể tiếp tục chương trình. Còn xảy ra trường hợp robot thoát ra khi chưa

khám phá hết toàn bộ môi trường, trả về bản đồ còn thiếu. Môi trường quá rộng so với robot nên thời gian thực hiện khảo sát rất lớn.

4.3.2. Cải thiện chương trình.

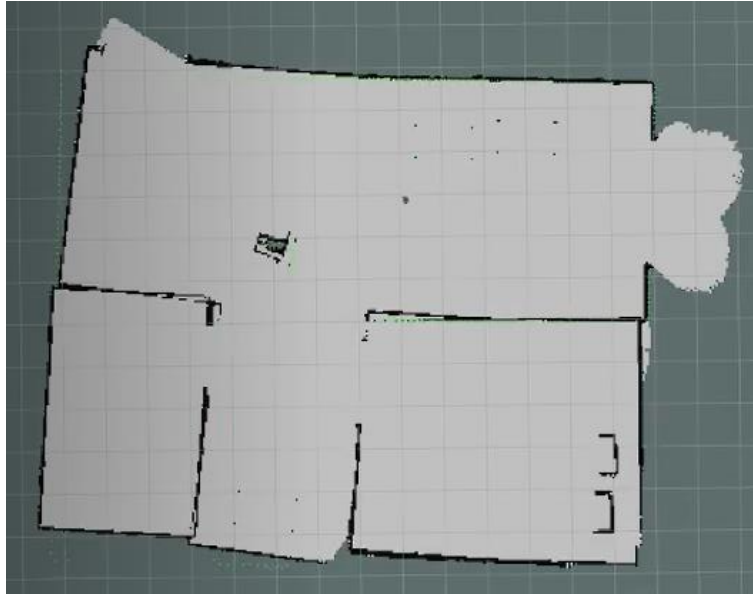
Mô tả sự cải thiện: Mục tiêu của bài là thu được bản đồ hoàn chỉnh cho toàn bộ các phòng. Robot biết vào phòng khi tiến hành thực hiện khảo sát môi trường chưa rõ và ra ngoài khi đã thu xong bản đồ. Có điều kiện đếm vòng lặp, có quỹ đạo di chuyển trong mỗi phòng, thoát khỏi phòng khi số vòng thực hiện quá số vòng lặp cho phép. Có điều kiện ngắt khi robot gặp sự cố và khi quá trình không đạt yêu cầu thời gian. Thực hiện ghép các mảnh bản đồ đã thu được sau mỗi lần bằng `merge_maps` để tạo bản đồ hoàn chỉnh, chấp nhận việc robot không thể đi hết bản đồ toàn bộ trong một lần thực hiện. Điều chỉnh tham số đầu vào của robot như trong phần tiêu chí cải thiện thuật toán. Chương trình tối ưu hóa quãng đường dịch chuyển của robot bằng một hàm xác định vị trí khám phá tiếp theo dựa vào thuật toán tính toán đường đi đến một điểm thuộc vùng chưa xác định, bằng cách tìm và đi theo các điểm không có vật cản khởi tạo được tạo thành một quỹ đạo đến vùng cần tìm. Từ đó giảm số vòng lặp tại một vị trí đã khám phá và tránh được tình trạng robot bị kẹt ở một địa điểm.



Hình 4.6. Quỹ đạo robot tạo ra để ra khỏi phòng.

Kết quả mô phỏng: Video kết quả mô phỏng được thực hiện [tại đây](#).

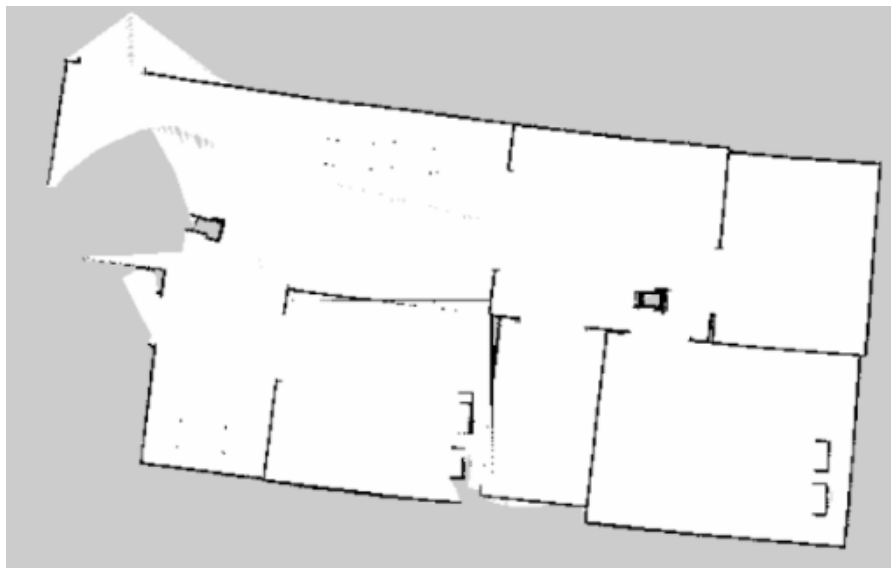
Các file kết quả thu được về hình ảnh, video quá trình và bộ dữ liệu thu được lập bản đồ được đặt ở [thư mục này](#) và [thư mục này](#). Hình ảnh dưới đây là bản đồ robot thu được sau quá trình thực hiện:



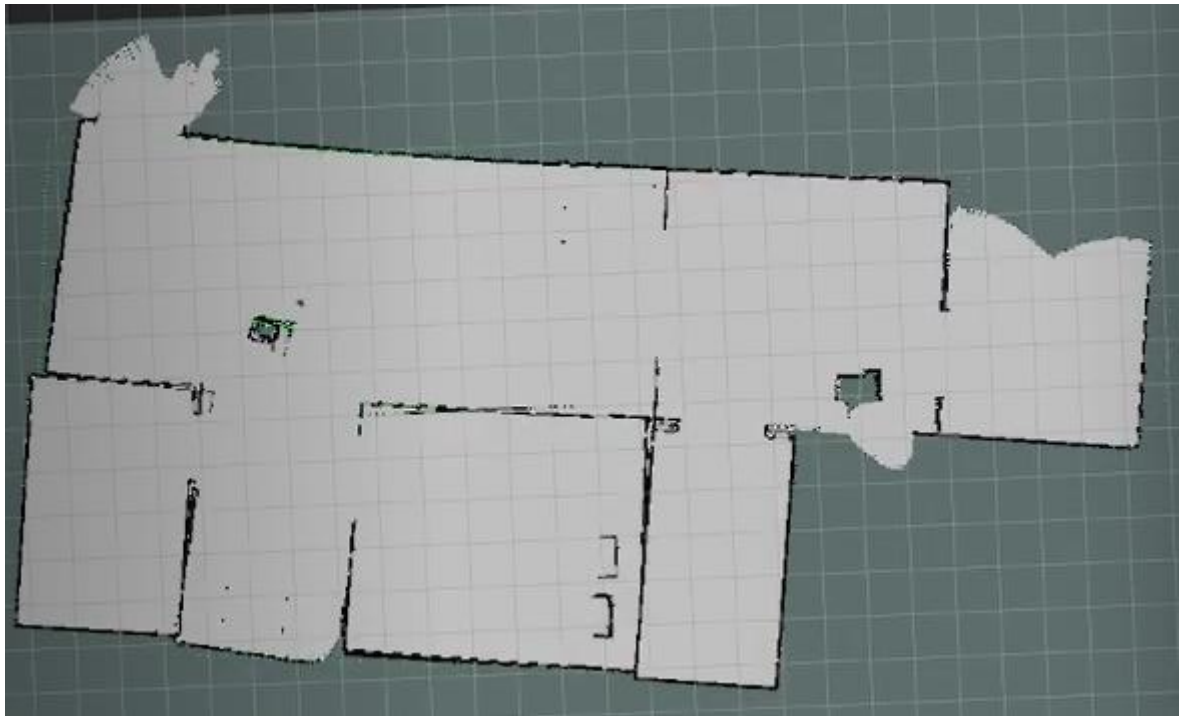
Hình 4.7. Bản đồ môi trường myhouse.world.

Nhận xét kết quả thu được: Quá trình thu được kết quả tốt hơn kết quả trước, thời gian mô phỏng ngắn hơn, phạm vi quét mở rộng hơn, trung bình mỗi lần thu được bản đồ với phạm vi rộng và chính xác hơn. Ta thực hiện merge_maps để kết hợp các phần bản đồ còn thiếu để tạo bản đồ hoàn chỉnh. Phạm vi hoạt động bao quát được toàn bộ khu vực bản đồ truyền vào môi trường.

Hình ảnh dưới đây là bản đồ robot thu được sau quá trình thực hiện:



Hình 4.8. Bản đồ môi trường myhouse.world.



Hình 4.9. Bản đồ môi trường lần chạy thử tư.



Hình 4.10. Kết quả bản đồ cuối cùng sau khi chạy chương trình tối ưu.

CHƯƠNG 5: TỔNG KẾT

Đề tài đã thành công xây dựng chương trình tự động định vị vị trí và dựng bản đồ môi trường trong không gian hai chiều cho robot di động trong nhà. Kết quả của quá trình thực hiện được đánh giá trên độ phức tạp của môi trường theo ba mức: Dễ - Trung bình - Nâng cao. Ta thực hiện tự động điều hướng và khảo sát trên ba môi trường này để ghi lại bản đồ bằng phương pháp Gmapping. Chương trình thực hiện định vị và điều hướng robot trong môi trường chưa biết bằng phương pháp tránh vật cản cơ bản và với quỹ đạo tối ưu hơn tránh di chuyển trong vòng lặp.

Chương trình hoàn thành mục tiêu cơ bản của đề tài - dựng thành công bản đồ môi trường xung quanh robot và đã thực hiện được một số sự cải tiến trong chương trình với sự tinh chỉnh các tham số trong chương trình làm việc. Hơn nữa, ta đã cải thiện chương trình mở rộng phạm vi quét của LiDar, thu khoảng cách nhận diện vật cản giúp robot di chuyển trong phạm vi đường đi hẹp hơn, mở rộng khả năng khảo sát môi trường của robot và giảm thời gian chạy chương trình. Triển khai thực hiện bài toán đã giải quyết được một số lỗi trong chương trình như chỉnh lại bản đồ bị lệch trong quá trình đo đạc, hợp nhất nhiều bản đồ của robot trong các quá trình khảo sát để dựng được bản đồ môi trường hoàn chỉnh.

Tuy nhiên, mô hình trên vẫn còn nhiều vấn đề có thể triển khai hướng tối ưu và cải thiện khác trong tương lai về tối ưu quỹ đạo khám phá, tinh chỉnh thông số bộ lọc để giảm sai lệch khi thu và dựng bản đồ. Chương trình được xây dựng trên môi trường mô phỏng vậy nên chưa thể khái quát chính xác hành vi của robot nếu thực hiện trong thực nghiệm. Để có thể áp dụng vào thực tế, ta cần phải triển khai mở rộng hơn nữa những thuật toán tối ưu vào chương trình để có thể đáp ứng được điều kiện phức tạp với nhiều biến số không xác định ảnh hưởng đến bài toán. Vậy, đề tài còn nhiều hướng phát triển sâu rộng hơn trong tương lai để cải thiện hiệu quả hoạt động, làm tiền đề cho áp dụng vào mô hình robot thực.

TÀI LIỆU THAM KHẢO

- [1] Chen C and Yinhang C 2008 Research on Map Building by Mobile Robots Proceedings - 2nd International Symposium on Intelligent Information Technology Application (IITA) 2 673–677
- [2] Wolf D. F. and Sukhatme G. S., 2008 Semantic mapping using mobile robots IEEE Trans. Robot 24(2) 245–258
- [3] Rusu R B 2010 Semantic 3D Object Maps for Everyday Manipulation in Human Living 612345678904th International Conference on Mechanical Engineering Research (ICMER2017) IOP Publishing IOP Conf. Series: Materials Science and Engineering 257 (2017) 012088 doi:10.1088/1757-899X/257/1/012088 Environments KI - Kunstl. Intelligenz 24 345–348
- [4] Zlot R and Bosse M 2014 Efficient large-scale 3D mobile mapping and surface reconstruction of an underground mine Springer Tracts Adv. Robot 92 479–494
- [5] Cortes B B, Sole X C and Salvi J 2011 Indoor SLAM using a range-augmented omnidirectional vision Proceedings of the IEEE International Conference on Computer Vision 280–287
- [6] Quigley M, Conley K, Gerkey B, FAust J, Foote T, Leibs J, Berger E, Wheeler R and Mg A 2009 ROS: an open-source Robot Operating System IEEE International Conference on Robotics and Automation
- [7] Cousins S 2010 ROS on the PR2 IEEE Robot. Autom. Mag 17 23–25
- [8] Labbe M and Michaud F 2014 Online Global Loop Closure Detection for Large-Scale Multi-Session Graph-Based SLAM IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) 2661–2666
- [9] Labbe M and Michaud F 2013 Appearance-Based Loop Closure Detection IEEE Trans. Robot 29(3) 734–745
- [10] Grisetti G, Grzonka S, Stachniss C, Pfaff P and Burgard W 2007 Efficient Estimation of Accurate Maximum Likelihood Maps in 3D IEEE/RSJ International Conference on Intelligent Robots and Systems 3472–3478
- [11] C. Georgoulas, T. Linner, A. Kasatkin, and T. Bock. An ami environment implementation: Em-bedding turtlebot into a novel robotic service wall. In ROBOTIK 2012; 7th German Conference on Robotics, pages 1–6, May 2012.

- [12] Monica Anderson LaPoint Maria Gini Nikolaos Papanikolopoulos John Budenske Steven Damer, Luke Ludwig. Dispersion and exploration algorithms for robots in unknown environments. volume 6230, 2006.
- [13] P Raja and S Pugazhenti. Optimal path planning of mobile robots: A review. International Journal of the Physical Sciences, 7:1314–1320, 02 2012.
- [14] Dora-Luz Almanza-Ojeda, Yazmín Gomar-Vera and Mario Ibarra-Manzano. Occupancy Map Construction for Indoor Robot Navigation. 10 2016.
- [15] Stan Franklin and Art Graesser. Is it an agent, or just a program?: A taxonomy for autonomous agents. In Proceedings of the Workshop on Intelligent Agents III, Agent Theories, Architectures, and Languages, ECAI '96, pages 21–35, London, UK, UK, 1997. Springer-Verlag.